MDPI

Article

Parameter-Efficient Fine-Tuning vs. Prompt Engineering for Point-of-Interest Recommendation: Lessons from Truncated Training Data

Luka Sever [1], Ratomir Karlović [1], Sandi Baressi Šegota [1,*], Vedran Mrzljak [2,*] and Ivan Lorencin [1]

[1] Faculty of Informatics, Juraj Dobrila University of Pula, Alda Negrija 6, 52100 Pula, Croatia; luka.sever@unipu.hr (L.S.), ratomir.karlovic@unipu.hr (R.K.), sandi.baressi.segota@unipu.hr (S.B.Š.), ivan.lorencin@unipu.hr (I.L.)
[2] Faculty of Engineering, University of Rijeka, Vukovarska 58, 51000 Rijeka, Croatia; vedran.mrzljak@riteh.uniri.hr (V.M.)
* Correspondence: sandi.baressi.segota@unipu.hr; vedran.mrzljak@riteh.uniri.hr

Abstract

Large language models (LLMs) can recommend points of interest (POIs) from a user profile and retrieved POI records, using prompt engineering or parameter-efficient fine-tuning (PEFT). This study compares both approaches for tourism recommendation in Pula, Croatia. Low-Rank Adaptation (LoRA) and Infused Adapter by Inhibiting and Amplifying Inner Activations (IA^3) adapters were trained on Llama 3.1 8B and Llama 3.2 3B, and six configurations, with and without adapters, generated recommendations for 1521 test queries. Three annotators labeled each recommendation, and their agreement was measured with Fleiss' kappa (κ). Training examples averaged about 1790 tokens but were truncated to 512 tokens, which removed the reference response in almost all of them. Agreement was almost perfect for all configurations (κ between 0.863 and 0.904). Every adapter configuration had a higher kappa than the same model without an adapter, but only LoRA on Llama 3.1 8B remained significant after Holm correction. The adapters reacted differently to the truncated training data: 19.7% and 73.4% of the LoRA responses contained copied JavaScript Object Notation (JSON) records, compared with at most 0.3% for IA^3, which received the highest share of agree labels (77.7% and 74.8%). The annotators gave the agree label in more than half of their ratings of responses with copied records. Agreement-based evaluation should therefore be combined with checks of the training sequence length and the output format.

Keywords: large language models; parameter-efficient fine-tuning; prompt engineering; point-of-interest recommendation; LoRA; IA3; inter-annotator agreement; Fleiss' kappa; sequence truncation; retrieval-augmented generation

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Tourists who visit a city for the first time need suggestions for places to eat, stay and visit that match their own interests and constraints, such as a budget, a minimum rating or a dislike of crowded places. Point-of-interest (POI) recommender systems address this need. Large language models (LLMs) make it possible to give such recommendations in natural language, based on an explicit user profile and on structured information about local POIs. Evaluating these recommendations is difficult, because whether a recommendation suits a user is partly subjective, and for a given request there is usually no single correct answer.

Recent work on LLM-based recommenders treats recommendation as a sequence generation task, and adapting these models to changing user preferences requires a balance between stability and plasticity. To address the continuous evolution of users and items, parameter-efficient fine-tuning (PEFT) methods such as Low-Rank Adaptation (LoRA) are used. For instance, the PESO framework maintains a single evolving adapter anchored by a proximal term, which captures new user behavior while reducing the forgetting associated with sequential updates [1].

Using LLMs as conversational recommenders introduces further problems, such as semantic misalignment between dialogue and recommendation and complex training processes. The PECRS framework reformulates conversational recommendation as a natural language processing task. It uses a frozen pre-trained language model and textual item metadata to perform both item recommendation and response generation, and it can be trained in a single stage with PEFT techniques, which reduces the computational cost [2].

Standard PEFT methods use one static LoRA module that is shared by all users, so the parameters are not personalized. The RecLoRA framework addresses this with a personalized LoRA module, which generates separate weights for each user through a soft routing mechanism guided by a conventional recommendation model [3].

Prompt engineering and in-context learning offer an alternative way to use LLMs without changing their weights. Because routing all users through an LLM is computationally expensive, a hybrid task allocation framework identifies "weak users" with sparse interaction histories and passes only their histories to an LLM as natural language instructions. This reduces token usage and latency while improving recommendation quality for the users that conventional systems serve poorly [4].

In multi-domain industrial settings, adaptation also has to balance general user behavior with domain-specific data sparsity. MLoRA combines a shared backbone, which learns information common to all domains, with low-rank modules that capture the features of each domain [5].

Most of these studies use interaction histories from benchmark datasets and measure accuracy against observed user behavior. Less is known about how adapter-based fine-tuning and prompt engineering compare when the user profile is explicit, the item data are structured JavaScript Object Notation (JSON) records, and no observed behavior is available as ground truth. The goal of this study is to compare two PEFT methods, LoRA and Infused Adapter by Inhibiting and Amplifying Inner Activations (IA^3), with a prompt engineering baseline for personalized POI recommendation in Pula, Croatia, on two foundation models, Llama 3.1 8B and Llama 3.2 3B. Because no objective ground truth exists, the study measures how consistently independent human annotators judge the recommendations produced by each configuration, using Fleiss' kappa (κ). The following hypotheses were tested:

- H1: For every configuration, the annotators reach at least substantial agreement ($\kappa > 0.60$).
- H2: For each foundation model, every adapter configuration has a higher kappa than the prompt engineering baseline of the same model ($\kappa_{\text{PEFT}} > \kappa_{\text{prompt}}$).
- H3: For each foundation model, the LoRA configuration has a higher kappa than the IA^3 configuration ($\kappa_{\text{LoRA}} > \kappa_{\text{IA}^3}$).
- H4: The order of the three adaptation strategies by kappa is the same for Llama 3.1 8B and Llama 3.2 3B.

H1 tests whether the annotation protocol gives reliable judgments for every adaptation method. H2 follows from the expectation that fine-tuning on accepted responses aligns the outputs more closely with explicit user preferences. H3 follows from the larger number

of parameters that LoRA trains, and H4 tests whether the results hold for two foundation models of different size and generation.

During the analysis of the training setup, we found that the maximum sequence length used for training (512 tokens) was much shorter than the training examples, so the reference response was cut off in almost all of them. Because this affected what the adapters could learn, the study also examined the generated responses for content copied from the retrieved POI records and for unfinished responses. The contributions of this study are:

1. A dataset of 5000 synthetic user profiles and queries for POI recommendation in Pula, with reference responses labeled by two annotators.
2. A comparison of prompt engineering, LoRA and IA^3 on two foundation models, evaluated through the agreement of three annotators on 1521 test queries.
3. An analysis of how training with truncated sequences affected the two adapter types, based on the content of their generated responses.
4. Evidence that annotator agreement and the share of accepted recommendations did not reveal responses with copied POI records, so agreement-based evaluation needs additional checks of the output.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the data, the adaptation strategies and the evaluation protocol. Section 4 presents the agreement results and the analysis of the generated responses, and discusses them. Section 5 describes the limitations and future work, and Section 6 concludes the paper.

2. Background and Related Work

Elmoghazy et al. proposed a taxonomy of LLM-based recommender systems based on the role of the LLM, the learning paradigm and the system architecture. They note that LLMs can act as text feature encoders or generate recommendations directly, but that they are limited by the context window, by latency, and by the difference between language modeling objectives and recommendation tasks [6]. Liu et al. reviewed how language modeling paradigms are adapted to recommendation, comparing pre-training with parameter-efficient prompt learning as ways to address data sparsity, and noted open problems with factual consistency and user privacy [7].

Nawara and Kashef reviewed how LLM-based systems address the data sparsity, cold-start and long-tail problems of conventional recommenders. Their survey groups recommenders into five paradigms (discriminative, generative, hybrid, graph-enhanced and multi-modal) and discusses the move from traditional metrics to generative evaluation and "LLM-as-a-Judge" frameworks [8]. Shehmir and Kashef reviewed more than 150 studies on the use of transformer-based models such as Bidirectional Encoder Representations from Transformers (BERT), Generative Pre-trained Transformer (GPT), and Text-to-Text Transfer Transformer (T5) in recommendation. They describe where prompt engineering and PEFT are used and point to remaining problems such as high inference latency and social bias [9].

For personalization, Chen et al. described how LLMs change recommendation from passive filtering into an active, conversational and explainable process. Their taxonomy describes LLMs as content interpreters, reasoners, conversational agents and controllers of external tools, and they note the cost of industrial deployment and the problem of hallucinated content [10]. Ling et al. reviewed how general-purpose LLMs are specialized for specific domains. They group adaptation techniques by how much access they need to the model: black-box external augmentation, grey-box prompt crafting and white-box fine-tuning. They note that fine-tuning aligns the model more closely with the domain, but brings a risk of catastrophic forgetting and a high resource cost [11].

Turning from surveys to empirical studies, Bao et al. proposed TALLRec, which tunes LLaMA-7B with LoRA in two stages, first on general instruction data and then

on recommendation data. In their experiments on movie and book recommendation, in-context learning alone gave weak results, while TALLRec improved the recommendation ability of the model with fewer than 100 training samples and generalized across domains [12]. Xu et al. proposed LLMRec to evaluate LLMs used directly as recommender systems. They showed that fine-tuning open-source models with parameter-efficient methods such as LoRA adapted them to the domain better than zero-shot use, and that reasoning prompts such as Chain-of-Thought still matter for prediction accuracy [13].

Among parameter-efficient methods, Zhu et al. addressed the limits of static LoRA matrices with RecLoRA. The framework combines several meta-LoRA weights into a personalized module for each user and balances recommendation quality against processing time [3]. Kim et al. found that foundation models fine-tuned with standard PEFT often produce item embeddings that do not depend on the user. To address this, they proposed PerPEFT, which groups users with similar interests and trains a separate PEFT module for each group. With group-specific hard negative sampling, it captures fine-grained item characteristics, and it works with several PEFT methods, including LoRA and IA^3, with few additional parameters [14]. Zeng et al. addressed changing preferences and the catastrophic forgetting caused by global fine-tuning. Their Region-Aware Incremental Editing framework clusters past interactions into knowledge regions. When new behavior appears, a confidence-aware routing mechanism updates only the LoRA module of the affected region, which balances learning new interests with keeping stable past behavior [15].

In domain-specific applications, Guo et al. used Quantized LoRA (QLoRA) within the TIGMI framework to fine-tune an LLM for cultural tourism. This approach reduced computational overhead by 60% while integrating Retrieval-Augmented Generation (RAG) for real-time multimodal interactions [16]. Similarly, Choi et al. applied LoRA to the upper layers of a BERT-based model to enhance a RAG framework for autonomous driving, which reduced the number of trainable parameters and improved the retrieval of complex legal texts [17]. Gao et al. developed a multi-dimensional framework that uses fine-tuned LLMs to assess the quality and reliability of large volumes of tourist-generated content on digital platforms [18].

Prompt engineering is an alternative that does not change the model weights. Nikolaychuk and Poddubniy applied prompt engineering to the Saiga2 model to automatically extract and classify user sentiments from regional tourism data, and reported high agreement with the evaluations of human experts [19]. To address errors in intent recognition and factual hallucinations, Huang et al. proposed a multi-layer prompt engineering architecture that decomposes user queries into sequential subtasks. By integrating this step-by-step reasoning strategy with real-time external application programming interfaces (APIs), their system reduced hallucination rates, especially for lightweight models in travel planning applications [20].

In summary, prior work has applied both PEFT and prompt engineering to recommendation and tourism tasks. Among the studies reviewed here, we did not find one that compares the two approaches on the same foundation models with explicit user profiles and structured POI records, and that evaluates the recommendations through the agreement of independent human annotators. This study addresses this gap, as described in Section 1.

3. Methodology

This section describes how PEFT and prompt engineering were compared for personalized POI recommendation. We outline the data collection process, the synthesis of user profiles and the annotation of reference responses, the RAG pipeline, the specific

adaptation strategies applied to the foundation models, and the evaluation protocol based on inter-rater agreement.

3.1. System Architecture Overview

The recommendation system uses RAG over structured POI data. The knowledge base, the FAISS index and the foundation models are the same for all configurations, so the configurations differ in how the foundation model is adapted: through prompt engineering only, or through a LoRA or IA^3 adapter. Two foundation models, Llama 3.1 8B and Llama 3.2 3B, are used to test whether the results hold for models of different size. Figure 1 shows the whole pipeline, from the preparation of the POI data to the evaluation of the recommendations.

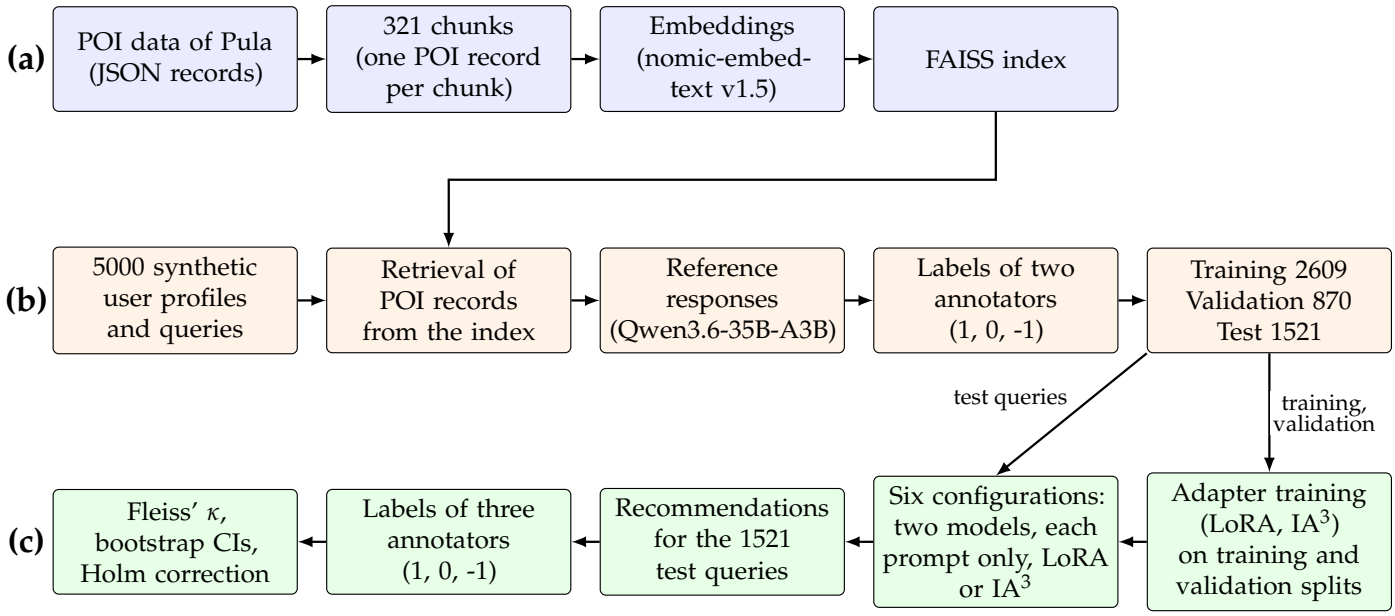


Figure 1. Pipeline of the study. (**a**) The POI records of Pula are split into chunks, embedded and stored in a FAISS index. (**b**) For each of the 5000 synthetic user profiles, POI records are retrieved for the user query, Qwen3.6-35B-A3B generates a reference response, and two annotators label it. The labels define the training, validation and test splits. (**c**) LoRA and IA^3 adapters are trained on the training and validation splits. For the test queries, the six configurations (each foundation model with prompt engineering only, with LoRA and with IA^3) generate recommendations, which three annotators label. Their agreement is measured with Fleiss' kappa.

3.2. Data Collection and Vector Indexing

A dataset of POIs in the city of Pula, Croatia, was collected by web scraping. The final knowledge base contains 321 POIs of 32 types. Table 1 shows the ten most frequent types. The 126 gastronomy POIs (restaurants, cafes, fast food places and pubs) are the largest group. The remaining POIs include accommodation, cultural heritage sites and recreational areas.

Table 1. Number of POIs in the knowledge base by type. The ten most frequent types are listed, and all remaining types are grouped as other.

Type	POIs	Share of all POIs (%)
Restaurant	74	23.1
Cafe	34	10.6
Artwork	24	7.5
Fort	18	5.6
Information	18	5.6
Park	16	5.0
Hotel	16	5.0
Fast food	15	4.7
Attraction	14	4.4
Apartment	14	4.4
Other	78	24.3
Total	321	100.0

The collected data were converted to a common JSON schema. Each POI is represented as a single JSON object containing a categorical type and a nested properties dictionary. Each record contains the following fields:

- Identification and Contact Fields: The name of the POI for all POIs, and, for gastronomy POIs only, its phone number, opening hours and address (street and house number).
- Taxonomy Tags: An array of descriptors defining the core attributes of the POI, such as cuisine (e.g., mediterranean), dishes (e.g., pizza), dietary options (e.g., vegetarian) and location (e.g., city_center). For some POIs, the tags are stored as a single text string instead of an array.
- Binary and Categorical Constraints: For gastronomy POIs, features such as outdoor_seating, indoor_seating, and price (low, medium or high) that describe the practical constraints of the POI.
- Contextual Narratives: A textual description, a numeric rating and a list of user reviews, which describe the opinions of visitors and the atmosphere of the place.

Not every field is available for every POI. Missing values are stored as null. Tags are available for 295 of the 321 POIs, descriptions for 254, ratings for 229 and user reviews for 222. Ratings are between 2.5 and 5.0. A POI has between one and five reviews, most often three. Among the 126 gastronomy POIs, the price is available for 91, the phone number for 108 and the opening hours for 111.

Each JSON object was treated as an independent document chunk, which gives 321 chunks. To enable dense retrieval, embeddings were generated with the nomic-embed-text v1.5 model. The resulting vectors were indexed with Facebook AI Similarity Search (FAISS) for similarity search.

3.3. Synthetic User Profile Generation and Annotation of Reference Responses

Evaluating personalized recommendations requires user profiles with varied intents and constraints. We synthetically generated 5000 unique user profiles. Each profile represents a distinct combination of demographic and behavioral traits, mapped to the JSON schema:

- Demographics: gender, age, and nationality to simulate cultural and generational variance.
- Behavioral Archetypes: travel_style (e.g., backpacker, luxury) and persona (e.g., pet_owner) to establish the user's primary motivation.
- Explicit Preferences: likes and dislikes structured as arrays of venue tags that describe what the user wants and wants to avoid.

- Quantitative Constraints: price_preference and min_rating that limit which POIs are acceptable.
- Natural Language Query (input): A conversational prompt synthesized from the profile's constraints (e.g., "Any nice outdoor spots where I can bring my pet?").

To obtain reference responses for training and evaluation, the Qwen3.6-35B-A3B model generated an initial recommendation response for each of the 5000 queries. Because personalized recommendation is subjective, two annotators checked whether each response met the explicit constraints of the user profile.

The annotators independently marked every response with 1 (agree), 0 (neither) or -1 (disagree). These two annotators were different from the three annotators who rated the outputs of the six configurations on the test split (Section 3.9). Agreement between the two annotators was low. Of the 5000 responses, 4995 received a valid label from both annotators. On these responses, Cohen's kappa [21] was -0.001, which is agreement no better than chance. One annotator marked 96.6% of the responses with 1 and the other 72.1%, and the two annotators gave the same label to 70.3% of the responses. Using only the responses that both annotators marked with 1 for training and validation is therefore a stricter selection than the judgment of either annotator alone. These labels were used to divide the dataset into training, validation and test splits, as described in Section 3.4.

3.4. Dataset Composition and Splits

The dataset is synthetic: the 5000 user profiles and their queries were generated and were not collected from real users. The dataset contains 5000 records, one for each synthetic user profile. Each record contains the profile fields (user identifier, name, age, gender, nationality, travel style, persona, likes, dislikes, price preference and minimum rating), the natural language query, the retrieved local information, the initial response generated by Qwen3.6-35B-A3B, and the annotator labels. The retrieved local information is a numbered list of POI records in the JSON format described above, each with its type and its properties. The records were divided into three splits that do not overlap, based on the labels given by the two annotators. Table 2 shows the number of queries and the selection rule for each split.

Table 2. Splits of the dataset.

Split	Queries	Share of dataset (%)	Selection rule	Use
Training	2609	52.2	Both annotators marked 1	Training of the adapters
Validation	870	17.4	Both annotators marked 1	Validation of the adapters
Test	1521	30.4	At least one of the two annotators did not mark 1	Evaluation of the six configurations

The 3479 queries that both annotators marked with 1 were divided between training and validation in a ratio of 75:25. The queries were ordered by user identifier: the first 2609 (identifiers 1–3795) formed the training split, and the remaining 870 (identifiers 3797–5000) formed the validation split. Because the identifiers were assigned in the random order in which the profiles were generated, an identifier carries no information about the profile. The shares of personas, travel styles and price preferences differ by at most 2.5 percentage points between the two splits.

The training and validation splits therefore contain only responses that both annotators accepted. The test split contains the remaining queries, whose responses at least one annotator did not accept. All six configurations were evaluated on the test split. The models without adapters did not use the training or validation splits.

3.5. Retrieval and Context Injection Pipeline

During inference, a standardized retrieval pipeline is maintained across all evaluated models to isolate the impact of the LLM adaptation strategy. For a given user input, the query is embedded and searched against the FAISS index. The five chunks with the smallest vector distance are retrieved, and these five JSON chunks are injected directly into the LLM prompt.

3.6. LLM Adaptation Strategies

The study compares a prompt engineering baseline with two PEFT methods:

- Prompt Engineering (Baseline): The foundation models receive the same system prompt and user prompt as the configurations with adapters (Section 3.8), but no adapter is loaded. It uses the five retrieved chunks through in-context learning, without changing the model weights.
- PEFT: The models are fine-tuned on the training split with two adapter types:
  - LoRA: Freezes the pre-trained model weights and adds trainable low-rank matrices to the layers of the transformer, so that only these matrices are trained.
  - IA^3: Scales inner activations with learned vectors and trains far fewer parameters than LoRA.

3.7. Adapter Training Configuration

Both adapter types were trained with the same procedure and the same hyperparameters. Each training example was formatted with the chat template of the foundation model and consisted of the system prompt, the user prompt (user profile, user question and retrieved local information) and the target response. The target response was the response generated by Qwen3.6-35B-A3B that both annotators had marked with 1. The 2609 queries of the training split were used to train the adapters, and the 870 queries of the validation split were used to monitor the training. The adapters were trained with the supervised fine-tuning trainer of the Transformer Reinforcement Learning (TRL) library and the PEFT library. Training used two NVIDIA A100 graphics processing units (GPUs). During training, the foundation models were loaded in bfloat16 precision without quantization. Table 3 lists the settings shared by both adapter types.

Table 3. Training settings shared by the LoRA and IA^3 adapters.

Setting	Value
Number of epochs	3
Learning rate	2×10^{-4}
Learning rate schedule	Linear decay after warmup
Warmup ratio	0.03
Optimizer	AdamW
Weight decay	0
Gradient clipping (maximum norm)	1.0
Batch size per GPU	1
Gradient accumulation steps	8
Number of GPUs	2
GPU type	NVIDIA A100
Effective batch size	16
Maximum sequence length (tokens)	512
Numerical precision	bfloat16
Gradient checkpointing	Yes
Random seed	42

The effective batch size was 16 examples, which is a batch size of 1 on each of the two NVIDIA A100 GPUs multiplied by 8 gradient accumulation steps. The validation loss was calculated every 50 steps, and a checkpoint was saved at the same interval. The checkpoint with the lowest validation loss was used as the final adapter. The length of the training examples was measured with the Llama 3 tokenizer, including the chat template. Because the POI records retrieved during training were not stored, the length of the retrieved records was estimated from the records that the same retrieval code returned for the 1521 test queries. On this basis, a training example contained on average about 1790 tokens, and the longest examples about 3400 tokens. Examples longer than the maximum sequence length of 512 tokens were truncated from the end. Even with the five shortest POI records in the knowledge base, 2605 of the 2609 training examples and 864 of the 870 validation examples would be longer than 512 tokens, so at least this many examples were truncated. In almost all examples, the system prompt, the user prompt and the retrieved records alone were longer than 512 tokens, so the reference response was not part of the truncated sequence. The loss was computed over the whole sequence, not only over the response. Table 4 lists the settings that are specific to each adapter type.

Table 4. Settings specific to each adapter type.

Setting	LoRA	IA^3
Rank	32	Not applicable
Scaling factor (alpha)	64	Not applicable
Dropout	0.05	Not applicable
Bias	None	Not applicable
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	k_proj, v_proj, down_proj
Feed-forward modules	Not applicable	down_proj

LoRA was applied to all attention and feed-forward projection layers of the transformer blocks, with trainable matrices of rank 32. IA^3 was applied only to the key, value and down projection layers. It trains one scaling vector for each of these layers, which rescales the outputs of the key and value projections and the input of the down projection,

so it has far fewer trainable parameters than LoRA. At inference, the trained adapters were applied to foundation models loaded with 4-bit quantization (Section 3.8).

Table 5 shows the number of trainable parameters, the training time and the final losses for the four adapters. The checkpoint with the lowest validation loss was the last checkpoint for all four adapters.

Table 5. Trainable parameters, training time, final training loss and validation loss of the four adapters. The training time is given in minutes.

Configuration	Trainable parameters	Training time (min)	Training loss	Validation loss
Llama 3.1 8B (LoRA)	83,886,080	28.5	0.088	0.102
Llama 3.1 8B (IA^3)	524,288	16.2	1.385	1.421
Llama 3.2 3B (LoRA)	48,627,712	24.5	0.094	0.108
Llama 3.2 3B (IA^3)	286,720	12.5	1.832	1.878

Training took between 12.5 and 28.5 minutes. LoRA has about 160 times more trainable parameters than IA^3 for Llama 3.1 8B and about 170 times more for Llama 3.2 3B. The loss values of the LoRA adapters (about 0.1) are much lower than those of the IA^3 adapters (1.4 and 1.9).

3.8. Prompt Format and Inference Setup

All six configurations received the same system prompt and the same user prompt format. The system prompt instructs the model to act as a tourism assistant that recommends personalized activities based on the interests in the user profile, the current question of the user, and the local information retrieved by semantic search. It also instructs the model to respect strong user preferences and to answer with a short and clear activity recommendation, followed by a brief explanation if needed. The user prompt contains the user profile (name, travel style, persona, likes, dislikes, price preference and minimum rating), the user question, and the retrieved local information as a numbered list of POI records. The full system prompt and the template of the user prompt are given in Appendix A.

At inference, all foundation models were loaded with 4-bit NormalFloat (NF4) quantization and double quantization, while the adapters had been trained on the foundation models loaded in bfloat16 without quantization (Section 3.7). For the configurations with adapters, the trained LoRA or IA^3 adapter was loaded on top of the quantized foundation model. The computations were done in bfloat16 for the prompt-only configurations and in float16 for the configurations with adapters. All recommendations were generated with greedy decoding, so the outputs do not depend on a sampling temperature or a random seed. For the prompt-only configurations, the response was limited to 512 new tokens. For the configurations with adapters, no limit on the number of new tokens was set, so generation stopped when the model produced its end-of-sequence token or reached its context length.

3.9. Evaluation Protocol and Inter-Rater Agreement

Six configurations were evaluated: Llama 3.1 8B and Llama 3.2 3B, each under prompt engineering, LoRA and IA^3. Each configuration is named after the foundation model, followed by the adaptation strategy in parentheses: Llama 3.1 8B (prompt only), Llama 3.1 8B (LoRA) and Llama 3.1 8B (IA^3), and the same for Llama 3.2 3B. Each configura-

tion generated recommendations for the same set of 1521 test queries. The same three annotators rated all recommendations. They were volunteer students of the Faculty of Informatics, Juraj Dobrila University of Pula, who know the city and have basic knowledge of computer science. They were not told which model or adaptation method produced each recommendation. The annotators were asked to judge whether each recommendation suited the user profile. For example, a user who dislikes meat and asks for a restaurant has to be recommended a place that serves meals without meat, and a user who likes a family environment has to be recommended places that are suitable for families. For every recommendation, the three annotators independently assigned one of three labels: 1 (the annotator agrees with the recommendation), -1 (the annotator disagrees) or 0 (neither). Since no objective ground truth exists for subjective recommendation quality, the evaluation relies on the consistency of these independent judgments, quantified with Fleiss' kappa [22].

Let N be the number of evaluated recommendations, $n = 3$ the number of annotators, $k = 3$ the number of categories, and n_{ij} the number of annotators who assigned recommendation i to category j. The observed agreement for a single recommendation and its mean are

$$P_i = \frac{1}{n(n-1)} \left(\sum_{j=1}^{k} n_{ij}^2 - n \right), \qquad \bar{P} = \frac{1}{N} \sum_{i=1}^{N} P_i. \tag{1}$$

The expected chance agreement is computed from the overall proportion p_j of ratings in each category, and Fleiss' kappa follows as

$$p_j = \frac{1}{Nn} \sum_{i=1}^{N} n_{ij}, \qquad \bar{P}_e = \sum_{j=1}^{k} p_j^2, \qquad \kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}. \tag{2}$$

A value of $\kappa = 1$ indicates perfect agreement, while $\kappa = 0$ indicates agreement no better than chance. Values are interpreted using the scale of Landis and Koch [23]: slight (0.01–0.20), fair (0.21–0.40), moderate (0.41–0.60), substantial (0.61–0.80) and almost perfect (0.81–1.00) agreement. Fleiss' kappa expresses how consistently annotators judge the recommendations, not whether the judgments are correct. Uncertainty was estimated by nonparametric bootstrap, resampling queries with replacement (20,000 replicates) and reporting 95% percentile intervals. Nine comparisons were planned: each adapter configuration against the same model without an adapter (four comparisons), LoRA against IA^3 for each model (two comparisons), and the same configuration (without adapter, with LoRA and with IA^3) on the two foundation models (three comparisons). The differences in kappa were estimated with a paired bootstrap on the same resampled queries. Because nine comparisons were made, the two-sided p-values were adjusted with the Holm method, which reduces the chance of false positive findings when several comparisons are tested together.

3.10. Analysis of the Generated Responses

After the annotation, the generated responses were checked automatically for two problems that the agreement measure does not capture. A response was counted as containing copied POI records if it contained one of the strings `"reviews"`, `"properties"`, `{"type"` or `]}`. These strings occur in the JSON records of the knowledge base but not in normal text. A response was counted as unfinished if its last character was not a period, a question mark, an exclamation mark, a closing bracket, an asterisk or a quotation mark. Both checks are simple text rules. They can miss some cases, and the second rule can count a response that ends with a list item as unfinished. For the configurations with copied

records, the share of label 1 was compared between the responses with and without copied records. These comparisons are descriptive and were not tested statistically.

4. Results and Discussion

This section reports the inter-rater agreement for the six configurations on the test set of 1521 queries. Three annotators rated each recommendation, which gives 4563 ratings for each configuration. The results are presented in five parts: the agreement for each configuration, the distribution of the labels, the comparison of the adaptation strategies, the analysis of the generated responses, and a discussion of the findings.

4.1. Agreement for Each Configuration

Table 6 shows Fleiss' kappa, its 95% confidence interval, the observed agreement $\bar{P}$ and the chance agreement $\bar{P}_e$ for each configuration. Figure 2 shows the kappa values.

Table 6. Fleiss' kappa (κ), 95% bootstrap confidence interval (CI), observed agreement ($\bar{P}$) and chance agreement ($\bar{P}_e$) for the six configurations on the 1521 test queries.

Configuration	κ	95% CI	$\bar{P}$	$\bar{P}_e$
Llama 3.1 8B (prompt only)	0.863	0.840 to 0.884	0.926	0.464
Llama 3.1 8B (LoRA)	0.902	0.882 to 0.920	0.950	0.487
Llama 3.1 8B (IA^3)	0.877	0.851 to 0.901	0.954	0.630
Llama 3.2 3B (prompt only)	0.875	0.854 to 0.895	0.930	0.439
Llama 3.2 3B (LoRA)	0.888	0.869 to 0.906	0.937	0.435
Llama 3.2 3B (IA^3)	0.904	0.882 to 0.924	0.961	0.597

Fleiss' kappa was between 0.863 and 0.904 for all six configurations. All values are in the almost perfect range (0.81 to 1.00) of the scale used in this study. The lower end of the 95% confidence interval was at least 0.84 for every configuration. Hypothesis H1 is therefore supported: the annotators reached at least substantial agreement under every adaptation strategy.

The observed agreement $\bar{P}$ was between 0.926 and 0.961. The chance agreement $\bar{P}_e$ was between 0.435 and 0.630. Chance agreement is the agreement that would be expected if the annotators gave their labels independently, with the label frequencies of the configuration. Kappa measures how much of the agreement above this level was reached. $\bar{P}_e$ was higher for the two IA^3 configurations (0.630 and 0.597) than for the other four configurations (0.435 to 0.487), because label 1 was more frequent for the IA^3 configurations (Section 4.2). For this reason, Llama 3.1 8B (IA^3) has the highest observed agreement of the three Llama 3.1 8B configurations (0.954), but a lower kappa (0.877) than Llama 3.1 8B (LoRA) (0.902), which has an observed agreement of 0.950.

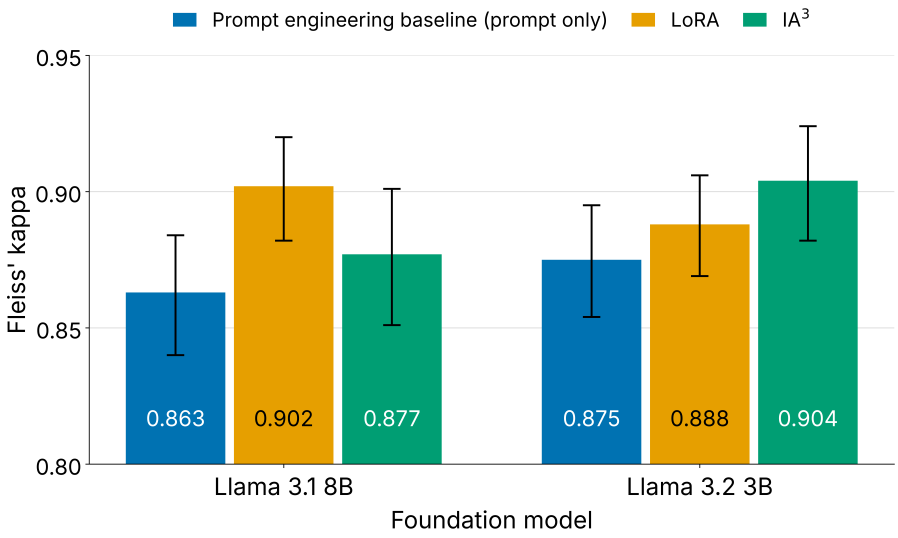


Figure 2. Fleiss' kappa for the six configurations on the 1521 test queries, grouped by foundation model. The prompt engineering baseline (prompt only) uses the foundation model without an adapter. The error bars show the 95% bootstrap confidence intervals from Table 6 (20,000 replicates, resampling queries). The value of kappa is printed inside each bar. The vertical axis starts at 0.80.

Figure 2 shows the same kappa values grouped by foundation model. The highest kappa was reached by Llama 3.2 3B (IA^3) (0.904), followed closely by Llama 3.1 8B (LoRA) (0.902), and the lowest by Llama 3.1 8B without an adapter (0.863). The differences between the configurations are small. The confidence intervals overlap for all configurations of the same foundation model. Overlapping intervals do not show that two configurations are equal, and the paired comparisons in Section 4.3 are used to test the differences between configurations, because they take into account that every configuration was rated on the same queries.

4.2. Distribution of the Labels

Table 7 shows how often each label was used. These counts are descriptive. They are reported to show what the annotators decided and to explain the chance agreement in Table 6.

Table 7. Number of ratings for each label and, in brackets, their share of the 4563 ratings given to each configuration (three annotators, 1521 queries), in percent.

Configuration	Label 1 (agree)	Label 0 (neither)	Label -1 (disagree)
Llama 3.1 8B (prompt only)	2720 (59.6)	389 (8.5)	1454 (31.9)
Llama 3.1 8B (LoRA)	2862 (62.7)	346 (7.6)	1355 (29.7)
Llama 3.1 8B (IA^3)	3544 (77.7)	389 (8.5)	630 (13.8)
Llama 3.2 3B (prompt only)	2534 (55.5)	441 (9.7)	1588 (34.8)
Llama 3.2 3B (LoRA)	2449 (53.7)	417 (9.1)	1697 (37.2)
Llama 3.2 3B (IA^3)	3414 (74.8)	337 (7.4)	812 (17.8)

Label 1 was given to 77.7% of the ratings for Llama 3.1 8B (IA^3) and to 74.8% for Llama 3.2 3B (IA^3). For the other four configurations, the share of label 1 was between 53.7% and 62.7%. Label -1 showed the opposite pattern: 13.8% and 17.8% for the two IA^3 configurations, and between 29.7% and 37.2% for the other four. Label 0 was used for between 7.4% and 9.7% of the ratings in every configuration. LoRA increased the share

of label 1 for Llama 3.1 8B (62.7% compared with 59.6%) but not for Llama 3.2 3B (53.7% compared with 55.5%).

These shares were not tested statistically, and Fleiss' kappa does not measure them. They show that the annotators agreed with the recommendations of the IA^3 configurations more often than with those of the other configurations. This may mean that the IA^3 recommendations matched the user profiles better, but this study did not test it.

4.3. Comparison of the Adaptation Strategies

Table 8 shows the nine planned comparisons. Each difference is the kappa of the configuration in the first column minus the kappa of the configuration in the second column, so a positive value means that the configuration in the first column has the higher kappa.

Table 8. Planned comparisons. Difference in Fleiss' kappa between the configuration in the first column and the configuration in the second column, its 95% bootstrap confidence interval (CI) and the p-value adjusted with the Holm method for nine comparisons. The confidence intervals are not adjusted for multiple comparisons.

Configuration	Compared with	Difference in κ	95% CI	Adjusted p
Llama 3.1 8B (LoRA)	Llama 3.1 8B (prompt only)	0.039	0.014 to 0.064	0.015
Llama 3.1 8B (IA^3)	Llama 3.1 8B (prompt only)	0.014	-0.016 to 0.044	0.878
Llama 3.2 3B (LoRA)	Llama 3.2 3B (prompt only)	0.013	-0.011 to 0.037	0.878
Llama 3.2 3B (IA^3)	Llama 3.2 3B (prompt only)	0.029	0.002 to 0.055	0.262
Llama 3.1 8B (LoRA)	Llama 3.1 8B (IA^3)	0.025	-0.003 to 0.053	0.458
Llama 3.2 3B (LoRA)	Llama 3.2 3B (IA^3)	-0.016	-0.034 to 0.003	0.458
Llama 3.1 8B (prompt only)	Llama 3.2 3B (prompt only)	-0.012	-0.037 to 0.012	0.878
Llama 3.1 8B (LoRA)	Llama 3.2 3B (LoRA)	0.014	-0.006 to 0.033	0.717
Llama 3.1 8B (IA^3)	Llama 3.2 3B (IA^3)	-0.027	-0.056 to 0.001	0.411

Hypothesis H2 expected that the models with adapters would be judged more consistently than the same models without adapters. All four adapter configurations had a higher kappa than the model without an adapter, with differences between 0.013 and 0.039. After the Holm correction, only the difference for LoRA on Llama 3.1 8B was statistically significant (0.039, 95% CI 0.014 to 0.064, adjusted p = 0.015). The difference for IA^3 on Llama 3.2 3B (0.029) had an unadjusted p-value of 0.033 and an adjusted p-value of 0.262. H2 is therefore only partially supported: all four differences have the expected direction, but only one of them is statistically significant after the Holm correction.

Hypothesis H3 expected that LoRA would reach a higher agreement than IA^3. LoRA had a higher kappa on Llama 3.1 8B (0.025) and a lower kappa on Llama 3.2 3B (-0.016). Neither difference was statistically significant (adjusted p = 0.458 for both). H3 is not supported.

Hypothesis H4 expected the same order of the strategies for both foundation models. The model without an adapter had the lowest kappa of its three configurations for both Llama 3.1 8B and Llama 3.2 3B. The order of LoRA and IA^3 was different for the two models. The same configuration did not differ significantly between the two foundation models: the differences (Llama 3.1 8B minus Llama 3.2 3B) were -0.012 without an adapter, 0.014 with LoRA and -0.027 with IA^3, with adjusted p-values between 0.411 and 0.878. The difference between the LoRA and IA^3 results was 0.041 larger for Llama 3.1 8B than for Llama 3.2 3B (95% CI 0.008 to 0.074). This comparison was not one of the nine planned comparisons and was not adjusted. H4 is therefore supported for the position of the model without an adapter, but not for the order of the two adapters.

4.4. Analysis of the Generated Responses

Table 9 shows how many responses of each configuration contained copied POI records and how many were unfinished, according to the checks described in Section 3.10.

Table 9. Responses with copied POI records and unfinished responses for each configuration, out of 1521 responses per configuration, with the share in percent in brackets. A response can have both problems, so the last column counts the responses with at least one of them.

Configuration	Copied POI records	Unfinished	At least one
Llama 3.1 8B (prompt only)	0 (0.0)	81 (5.3)	81 (5.3)
Llama 3.1 8B (LoRA)	300 (19.7)	382 (25.1)	410 (27.0)
Llama 3.1 8B (IA^3)	0 (0.0)	258 (17.0)	258 (17.0)
Llama 3.2 3B (prompt only)	0 (0.0)	119 (7.8)	119 (7.8)
Llama 3.2 3B (LoRA)	1116 (73.4)	880 (57.9)	1148 (75.5)
Llama 3.2 3B (IA^3)	4 (0.3)	157 (10.3)	157 (10.3)

No response of the two prompt-only configurations contained copied POI records. Copied records appeared in 19.7% of the responses of Llama 3.1 8B (LoRA) and in 73.4% of the responses of Llama 3.2 3B (LoRA), but in none of the responses of Llama 3.1 8B (IA^3) and in only 4 responses of Llama 3.2 3B (IA^3). Unfinished responses were least frequent for the prompt-only configurations (5.3% and 7.8%), more frequent for the IA^3 configurations (17.0% and 10.3%), and most frequent for the LoRA configurations (25.1% and 57.9%). For Llama 3.2 3B (LoRA), 848 of the 1116 responses with copied records were also unfinished.

The annotators often agreed with responses that contained copied records. For Llama 3.1 8B (LoRA), label 1 was given to 56.8% of the ratings of responses with copied records and to 64.2% of the ratings of responses without them. For Llama 3.2 3B (LoRA), the shares were 51.1% and 60.7%. The unfinished responses of the IA^3 configurations received label 1 in 76.6% (Llama 3.1 8B) and 70.3% (Llama 3.2 3B) of the ratings.

4.5. Discussion

The annotators reached almost perfect agreement for all six configurations. The test set contained only queries for which the annotators had not all marked the initial response with 1 during the annotation of the reference responses. Even so, the observed agreement for the recommendations of the six configurations was between 0.926 and 0.961. The annotators did not know which configuration produced each recommendation, so the agreement does not depend on knowledge of the model.

The differences in kappa between configurations were small, at most 0.041. No adapter configuration had a lower kappa than the same model without an adapter. Using LoRA or IA^3 therefore did not make the recommendations harder to judge consistently. The only difference that remained statistically significant after correction was LoRA on Llama 3.1 8B. There was no consistent difference between LoRA and IA^3, because the better adapter was not the same for the two foundation models. The same configuration also did not differ significantly between Llama 3.1 8B and Llama 3.2 3B.

The results for the IA^3 configurations show that kappa should be read together with the label distribution. These configurations received label 1 more often, which raised the chance agreement and lowered kappa relative to the observed agreement. Kappa measures how consistently the annotators judged the recommendations. It does not measure how accurate the recommendations are, so a higher kappa for one configuration does not show that its recommendations are better. The label shares in Table 7 point towards the IA^3 configurations as the ones whose recommendations the annotators accepted most often. A statistical test of the label shares is needed to confirm this.

The label shares also show a result that needs an explanation. The LoRA configurations received label 1 less often than the IA^3 configurations, although LoRA trains about 160 to 170 times more parameters, and Llama 3.2 3B (LoRA) received label 1 less often (53.7%) than the same model without an adapter (55.5%).

The most likely explanation is the truncation of the training examples (Section 3.7). The examples averaged about 1790 tokens, but only the first 512 tokens were used, and in almost all examples these tokens contained only the system prompt, the user prompt and part of the retrieved POI records. The reference response was therefore almost never part of the training text. Because the loss was computed over the whole sequence, the adapters were trained to continue the prompt and the POI records rather than to write a recommendation. The responses show the result of this training (Table 9). LoRA, which changes the weights of all seven projection layers, learned to reproduce the JSON records: 73.4% of the responses of Llama 3.2 3B (LoRA) and 19.7% of those of Llama 3.1 8B (LoRA) contained copied records. IA^3, which only rescales the activations of three layer types, almost never did. The training losses agree with this explanation. The LoRA adapters reached a loss of about 0.1 on the truncated sequences, which consist mostly of the same system prompt and of records of the same 321 POIs, while the IA^3 adapters stayed at 1.4 and 1.9 (Table 5). The validation examples were truncated in the same way, so the validation loss could not show that the adapters did not learn to write responses.

Under these training conditions, IA^3 appears to have kept the ability of the foundation models to write a recommendation, and its configurations received label 1 most often. LoRA changed the behavior of the foundation models more strongly, and for the smaller model this change came with a lower share of label 1 than without an adapter. These results describe adapters trained on truncated sequences. They do not show how LoRA and IA^3 compare when the full reference responses are part of the training text, and the study cannot separate the effect of the adapter type from the effect of the truncation.

Two other factors may have contributed. First, the adapters were trained on the foundation models in bfloat16 precision and applied to the same models with 4-bit quantization (Section 3.8). LoRA adds a learned update to the weights, which may be affected more by this difference than the scaling vectors of IA^3. Second, the training and validation splits contain only queries whose reference responses both annotators accepted, while the test split contains the remaining queries, so the adapters were tested on a different kind of query than they were trained on. Neither factor explains the copied POI records, which appear only in the adapter configurations.

The analysis of the responses also shows a limit of the evaluation by agreement. The annotators gave label 1 to 56.8% and 51.1% of the ratings of LoRA responses that contained copied POI records. The annotators were asked to judge whether a recommendation suited the user profile, and a response that names a suitable place and then continues with copied records can meet this condition. High agreement and a high share of label 1 therefore do not show that a response is usable in an application. An automatic check of the output, such as the checks in Section 3.10, adds information that the human labels did not capture.

Three practical lessons follow. First, the length of the training examples should be measured with the tokenizer of the model before training, and the maximum sequence length should cover the full example, including the response. Prompts that contain retrieved JSON records are long, and in this study the prompt alone was longer than 512 tokens in almost all examples. Second, the loss should be computed only on the tokens of the response, so that a truncated example does not train the adapter on the prompt. Third, the generated responses should be checked for format problems in addition to the human evaluation.

Some limits apply to these results. The recommendations are for one city, Pula, and were rated by three annotators. The test set contains only the queries that were difficult during annotation, and the label shares were not tested statistically. These and other limits are discussed in Section 5.

5. Limitations and Future Work

This section describes the limitations of the study and proposes directions for future work. The limitations concern the evaluation measure, the data, the models and training, and the settings used for inference. The future work follows from these limitations.

5.1. Limitations

The most important limitation is the truncation of the training examples. The maximum sequence length of 512 tokens was much shorter than the training examples, which averaged about 1790 tokens, so the reference response was cut off in almost all of them (Section 3.7). The adapters were therefore trained mostly on the prompt and the retrieved records, not on the reference responses. With a maximum sequence length that covers the full examples and a loss computed only on the response, the adapters may behave differently, and the differences between LoRA, IA^3 and prompt engineering may change in size or direction. The adapters were not retrained to test this. The checks for copied records and unfinished responses are simple text rules and may miss some cases.

The evaluation in this study is based on Fleiss' kappa, which measures how consistently the three annotators judged the recommendations. A high kappa shows that the annotators agreed with each other. It does not show that a recommendation is correct or that its quality is high. Personalized recommendation is subjective, and no objective ground truth exists. The shares of the labels in Table 7 were reported only as descriptive counts and were not tested statistically. For this reason, the study cannot state which configuration gives better recommendations.

Only responses that both annotators marked with 1 were used for training and validation. The agreement between these two annotators was no better than chance (Cohen's $\kappa =$ -0.001, Section 3.3), so the selection mainly reflects the judgment of the stricter annotator, and the accepted responses represent the views of two individuals. All training responses were also generated by one model, Qwen3.6-35B-A3B, so the adapters learned from the content and style of a single model. The test set of 1521 queries was used to test the configurations after training. It contains the queries for which the annotators did not all mark the initial response with 1, so these queries may be more difficult and more subjective than the rest of the dataset. The validation split was selected with the same rule as the training split, so the validation loss used to select the checkpoints shows how well an adapter reproduces accepted responses, but not how well it handles difficult queries. The results therefore show how the configurations behave on difficult cases, and they may be different for a random sample of queries.

The user profiles and queries are synthetic. Real users may write their requests in a different way, with different wording, length and level of detail, so the results may not transfer directly to real requests. The knowledge base covers one small city, Pula, with 321 POIs, and the system was not tested on a large city with many more POIs. Each POI is stored as one JSON object that contains all of its fields, and this design may not work as well for a much larger number of POIs. The results could be different for larger cities for this reason.

Only two foundation models were used, Llama 3.1 8B and Llama 3.2 3B, and both belong to the same model family. Each adapter was trained once, with one random seed and one set of hyperparameters, which was the same for LoRA and IA^3. The settings

were not tuned separately for each adapter type, so a different setting could favor one adapter type. The bootstrap confidence intervals describe the variation between test queries. They do not describe the variation between training runs, which can only be measured by training the same adapter several times.

All six configurations were evaluated with 4-bit quantization, while the adapters were trained on models loaded without quantization. Quantization changes the model weights slightly, so an adapter may behave differently on a quantized model than on the model it was trained on. An evaluation without quantization was not done, and the results may be different at full precision. Finally, the study did not measure inference latency or cost. The training time and the number of trainable parameters were reported, but the time and cost needed to generate recommendations were not compared between the configurations.

5.2. Future Work

Several directions follow from these limitations. The first and most direct is to retrain both adapter types with a maximum sequence length that covers the full training examples (up to about 3400 tokens in this dataset) and with the loss computed only on the response, and to repeat the evaluation with the same annotators. This would show how LoRA and IA^3 compare when they learn from the reference responses. The second is to test more foundation models, including models from other families and of different sizes, to check whether the results are specific to the two Llama models. The third is to build knowledge bases for other cities, including large cities with many more POIs, and to include more POI types. This would show whether storing each POI as one JSON object and retrieving five chunks still works at a larger scale.

The fourth direction is to train each adapter several times with different random seeds, so that the variation between training runs can be measured and reported together with the variation between test queries. The fifth is to evaluate the configurations with real user profiles and real questions written by people, instead of synthetic data. The sixth is to tune the hyperparameters, such as the rank, the learning rate and the number of epochs, separately for each adapter type, and to evaluate the adapters without quantization. Finally, the human evaluation can be combined with automatic checks of the output format and with a statistical test of how often the annotators accept the recommendations of each configuration, which would add a measure of recommendation quality to the measure of agreement used here.

6. Conclusions

This study compared two parameter-efficient fine-tuning methods, LoRA and IA^3, with prompt engineering for personalized point-of-interest recommendation in the city of Pula. Llama 3.1 8B and Llama 3.2 3B were used, which gave six configurations. Three annotators rated the recommendations of each configuration for 1521 test queries, without knowing which configuration produced them. Their agreement, measured with Fleiss' kappa, was between 0.863 and 0.904 for all six configurations, which is almost perfect agreement. All four adapter configurations had a higher kappa than the same model without an adapter, but the differences were small, and only Llama 3.1 8B (LoRA) remained statistically significant after correction for multiple comparisons.

The training examples were much longer than the maximum sequence length of 512 tokens, so the reference responses were cut off in almost all of them, and the adapters were trained mostly on the prompt and the retrieved POI records. The two adapter types reacted differently to this. LoRA learned to reproduce the records: 19.7% and 73.4% of its responses contained copied JSON records. IA^3 almost never did, and its configurations received label 1 most often (77.7% and 74.8% of the ratings). The annotators still gave label 1 to more than

half of the ratings of LoRA responses with copied records, so agreement and acceptance alone did not reveal this problem.

For fine-tuning on prompts with retrieved records, the length of the training examples should be checked against the maximum sequence length, the loss should be computed only on the response, and the human evaluation should be combined with automatic checks of the output. The study used synthetic profiles and queries, one small city, two Llama models and one training run for each adapter. These limitations and the planned future work are described in Section 5.

Author Contributions: Conceptualization, R.K., L.S. and S.B.Š.; methodology, L.S.; software, L.S.; validation, I.L. and S.B.Š.; formal analysis, R.K.; investigation, R.K. and L.S.; resources, R.K. and V.M.; data curation, R.K. and L.S.; writing—original draft preparation, R.K. and L.S.; writing—review and editing, S.B.Š., I.L., and V.M.; visualization, R.K.; supervision, S.B.Š. and I.L.; project administration, S.B.Š. and I.L.; funding acquisition, V.M. and I.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0158 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136.

Institutional Review Board Statement: The annotators were volunteer students of the Faculty of Informatics, Juraj Dobrila University of Pula, who participated as annotators of model outputs rather than as research participants. No personal or sensitive data about the annotators were collected, and the user profiles in the dataset are synthetic. No institutional review board approval was required for this study under the applicable national regulations.

Informed Consent Statement: Informed consent was obtained from all annotators involved in the study. The annotators participated voluntarily and were not compensated.

Data Availability Statement: The original data presented in this study are openly available in a public repository at https://github.com/FIPU-SPIN/lora-tourism-recommender

Acknowledgments: This research was performed using the Advanced computing service provided by the University of Zagreb University Computing Centre (SRCE).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AdamW	Adaptive Moment Estimation with Decoupled Weight Decay
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
bfloat16	Brain Floating Point 16-bit Format
CI	Confidence Interval
EC	European Commission
EDIH	European Digital Innovation Hub
FAISS	Facebook AI Similarity Search
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
IA^3	Infused Adapter by Inhibiting and Amplifying Inner Activations
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
NF4	4-bit NormalFloat
PEFT	Parameter-Efficient Fine-Tuning
POI	Point of Interest
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
SRCE	University Computing Centre (Croatian: Sveučilišni računski centar)
T5	Text-to-Text Transfer Transformer
TRL	Transformer Reinforcement Learning
κ	Kappa coefficient of agreement (Fleiss' kappa, unless Cohen's kappa is stated)

Appendix A. Prompts

This appendix gives the system prompt and the template of the user prompt that were used for training the adapters and for generating the recommendations of all six configurations. In the user prompt, the text in angle brackets is replaced by the values from the user profile, the user question and the retrieved POI records.

System prompt

```
You are an intelligent tourism assistant.  You recommend personalized activities
to tourists based on:
1.  The user's interests and preferences (as defined in their profile).
2.  Their current intent or question (described in the user input).
3.  Relevant local information retrieved via semantic search (provided as bullet
points).
Your response should be personalized, context-aware, and practical.
If there are strong user preferences, make sure to respect them.
Respond in a friendly and informative tone.  Your output should be a short and
clear activity recommendation, followed by a brief explanation if needed.
```

User prompt template

```
User profile:
Name: <name>
Travel style: <travel style>
Persona: <persona>
Likes: <likes, separated by commas, or "none specified">
Dislikes: <dislikes, separated by commas, or "none specified">
Price preference: <price preference>
Minimum rating: <minimum rating>

User question: <user question>

Retrieved local information:
<retrieved POI records in JSON format, one record per line>
```

References

1. Yoo, H.; Li, T.W.; Kang, S.; Liu, Z.; Xu, C.; Qi, Q.; Tong, H. Continual Low-Rank Adapters for LLM-based Generative Recommender Systems. *arXiv preprint arXiv:2510.25093* **2025**. https://doi.org/10.48550/arXiv.2510.25093.
2. Ravaut, M.; Zhang, H.; Xu, L.; Sun, A.; Liu, Y. Parameter-efficient conversational recommender system as a language processing task. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 152–165. https://doi.org/10.18653/v1/2024.eacl-long.9.
3. Zhu, J.; Lin, J.; Dai, X.; Chen, B.; Shan, R.; Zhu, J.; Tang, R.; Yu, Y.; Zhang, W. Lifelong Personalized Low-Rank Adaptation of Large Language Models for Recommendation. *arXiv preprint arXiv:2408.03533* **2024**. https://doi.org/10.48550/arXiv.2408.03533.
4. Kaur, K.; Gupta, V.; Chadha, M.; Shah, C. Efficient and responsible adaptation of large language models for robust top-k recommendations. *ACM Transactions on Recommender Systems* **2026**, *4*, 1–31. https://doi.org/10.1145/3774778.
5. Yang, Z.; Gao, H.; Gao, D.; Yang, L.; Yang, L.; Cai, X.; Ning, W.; Zhang, G. Mlora: Multi-domain low-rank adaptive network for ctr prediction. In Proceedings of the 18th ACM Conference on Recommender Systems, 2024, pp. 287–297. https://doi.org/10.1145/3640457.3688134.
6. Elmoghazy, S.S.; Shouman, M.A.; Elminir, H.K.; Selim, G.E.I. Comparative analysis of methodologies and approaches in recommender systems utilizing large language models. *Artificial Intelligence Review* **2025**, *58*, 207. https://doi.org/10.1007/s10462-025-11189-8.
7. Liu, P.; Zhang, L.; Gulla, J.A. Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems. *Transactions of the Association for Computational Linguistics* **2023**, *11*, 1553–1571. https://doi.org/10.1162/tacl_a_00619.
8. Nawara, D.; Kashef, R. A comprehensive survey on LLM-powered recommender systems: from discriminative, generative to multi-modal paradigms. *IEEE Access* **2025**, *13*, 145772–145798. https://doi.org/10.1109/ACCESS.2025.3599832.
9. Shehmir, S.; Kashef, R. LLM4Rec: a comprehensive survey on the integration of large language models in recommender systems—approaches, applications and challenges. *Future Internet* **2025**, *17*, 252. https://doi.org/10.3390/fi17060252.
10. Chen, J.; Liu, Z.; Huang, X.; Wu, C.; Liu, Q.; Jiang, G.; Pu, Y.; Lei, Y.; Chen, X.; Wang, X.; et al. When large language models meet personalization: Perspectives of challenges and opportunities. *World wide web* **2024**, *27*, 42. https://doi.org/10.1007/s11280-024-01276-1.
11. Ling, C.; Zhao, X.; Lu, J.; Deng, C.; Zheng, C.; Wang, J.; Chowdhury, T.; Li, Y.; Cui, H.; Zhang, X.; et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys* **2025**, *58*, 1–39. https://doi.org/10.1145/3764579.
12. Bao, K.; Zhang, J.; Zhang, Y.; Wang, W.; Feng, F.; He, X. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In Proceedings of the 17th ACM Conference on Recommender Systems, 2023, pp. 1007–1014. https://doi.org/10.1145/3604915.3608857.
13. Xu, L.; Zhang, J.; Li, B.; Wang, J.; Chen, S.; Zhao, W.X.; Wen, J.R. Tapping the potential of large language models as recommender systems: A comprehensive framework and empirical analysis. *ACM Transactions on Knowledge Discovery from Data* **2025**, *19*, 1–51. https://doi.org/10.1145/3726871.
14. Kim, S.; Hwang, H.; Shin, K. Personalized Parameter-Efficient Fine-Tuning of Foundation Models for Multimodal Recommendation. In Proceedings of the ACM Web Conference 2026, 2026, pp. 6079–6090. https://doi.org/10.1145/3774904.3792248.
15. Zeng, J.; Qi, Y.; Li, H.; Li, C.; Lyu, Z.; Cui, L.; Bai, L. RAIE: Region-Aware Incremental Preference Editing with LoRA for LLM-based Recommendation. In Proceedings of the ACM Web Conference 2026, 2026, pp. 4038–4046. https://doi.org/10.1145/3774904.3792451.

16. Guo, H.; Liu, Z.; Tang, C.; Zhang, X. An interactive framework for personalized navigation based on metacosmic cultural tourism and large model fine-tuning. *IEEE Access* **2025**, *13*, 81450–81461. https://doi.org/10.1109/ACCESS.2025.3565359.
17. Choi, Y.; Kim, S.; Bassole, Y.C.F.; Sung, Y. Enhanced retrieval-augmented generation using low-rank adaptation. *Applied Sciences* **2025**, *15*, 4425. https://doi.org/10.3390/app15084425.
18. Gao, J.; Peng, P.; Claramunt, C.; Lu, F. Quality Assessment of Tourist Generated Contents: A Large Language Model Approach. In Proceedings of the Web and Wireless Geographical Information Systems; Mata-Rivera, M.F.; Zagal-Flores, R.; Siabato, W., Eds., Cham, 2026; pp. 31–41. https://doi.org/10.1007/978-3-032-01723-9_3.
19. Nikolaychuk, O.A.; Poddubniy, I.A. Analysis of Reviews of Tourist Objects Based on Large Language Models. In Proceedings of the 2024 IEEE 3rd International Conference on Problems of Informatics, Electronics and Radio Engineering (PIERE). IEEE, 2024, pp. 1070–1074. https://doi.org/10.1109/PIERE62470.2024.10805064.
20. Huang, Y.; Ma, L.; Wang, Y. Multi-Layer Prompt Engineering for Intent Recognition in Travel Planning Assistants. *Applied Sciences* **2025**, *15*, 11442. https://doi.org/10.3390/app152111442.
21. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* **1960**, *20*, 37–46. https://doi.org/10.1177/001316446002000104.
22. Fleiss, J.L. Measuring nominal scale agreement among many raters. *Psychological Bulletin* **1971**, *76*, 378–382. https://doi.org/10.1037/h0031619.
23. Landis, J.R.; Koch, G.G. The measurement of observer agreement for categorical data. *Biometrics* **1977**, *33*, 159–174. https://doi.org/10.2307/2529310.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.